慶應義塾 Keio University

DAY 4

# Today is conceptual design practice

## SysML v2 for product concepts

User intent / Product requirements / Structure and behavior

**Laurent Balmelli · Hidekazu Nishimura**
Keio University | October 2026

TODAY'S CONCEPT PRACTICE

01 **FRAME**
Clarify the user need

02 **MODEL**
Express intent in SysML v2

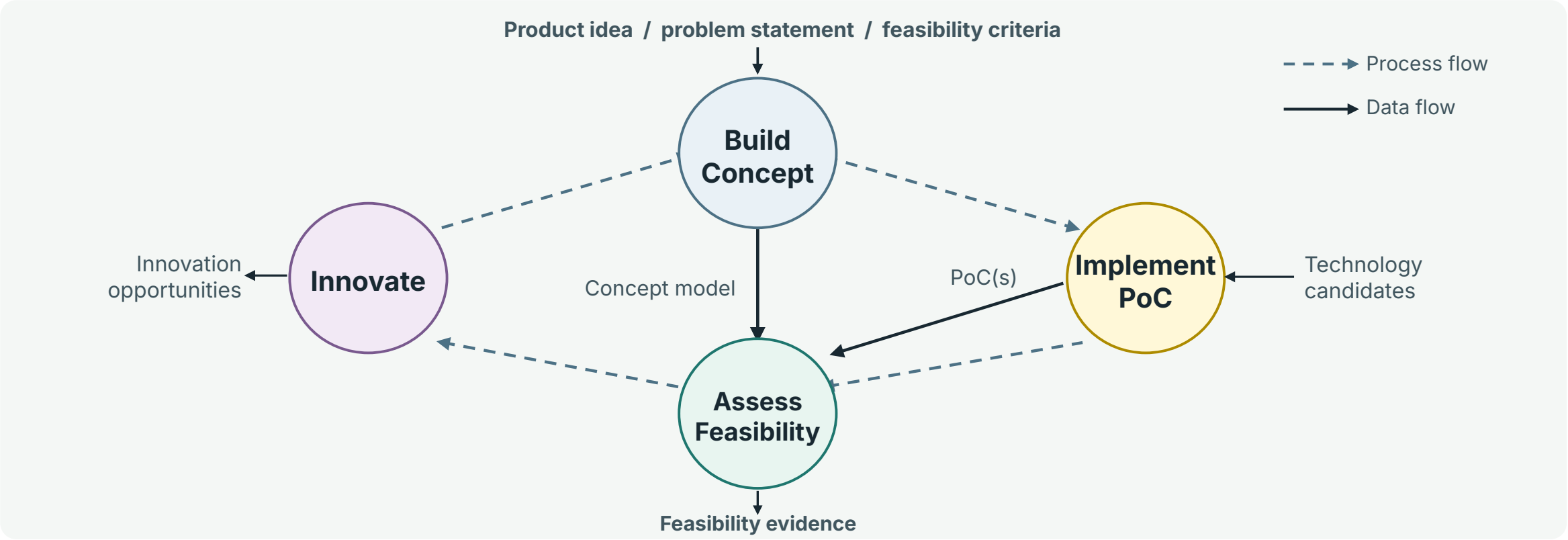
03 **REVIEW**
Explain assumptions and choices

SECTION 4.1.3 / THE DESIGN LOOP

4 MIN

# Visual Concept Making is an iterative loop

The concept, realizations and feasibility criteria are refined together.



SECTION 4.2.1 / MODELING PERSPECTIVES

2 MIN

# From product concepts to SysML v2

One product concept can be described through several connected modeling perspectives.

1

01 / REQUIREMENTS

## What must it do?

Capture ideas from anywhere

2

02 / STRUCTURE

## What is it made of?

Capture / Store / Display / Analyze

3

03 / BEHAVIOR

## How does it work?

Capture → Store → Display

4

04 / SPECIFICATIONS

## How well must it work?

Capacity and response criteria

ONE IDEA WALL CONCEPT MODEL

**Different elements, consistent meaning and traceability**

SECTION 4.2.2 / REQUIREMENTS

3 MIN

# Modeling requirements: the Idea Wall

Convert product assumptions into explicit requirements and feasibility criteria.

**IDEA WALL / REQUIREMENT DEFINITIONS**

```
package IdeaWallRequirements {
  private import ScalarValues::*;
  requirement def <'R1'> SharedIdeaCapture {
    doc /* Contributors shall capture ideas from
           anywhere and share them on
           collaborative team boards. */
  }
  requirement def <'FC1'> MultipleFormCapture {
    doc /* The system shall support capturing
           ideas in at least two formats,
           including text. */
    attribute supportedFormatCount : Integer;
    require constraint { supportedFormatCount >= 2 }
  }
  requirement sharedCapture : SharedIdeaCapture;
  requirement multipleFormCapture : MultipleFormCapture;
}
```

**FROM ASSUMPTION TO REQUIREMENT**

### R1 SharedIdeaCapture

A clear statement of intended capability

### FC1 MultipleFormCapture

A minimum number of supported formats

**FORMAL: at least two formats**
**PROSE: one must be text**

SECTION 4.2.3 / REQUIREMENTS PRIMITIVES

2 MIN

# SysML v2 primitives: requirements

Every highlighted expression in the preceding example has a specific modeling role.

| PRIMITIVE | DEFINITION | IDEA WALL EXAMPLE |
|---|---|---|
| **package** | Organizes model elements in a namespace | package IdeaWallRequirements |
| **import** | Makes external definitions available | private import ScalarValues::* |
| **requirement def** | Defines a reusable requirement | requirement def SharedIdeaCapture |
| **<'R1'>** | Short identifier for model traceability | <'R1'> |
| **doc** | Human-readable intent attached to a model element | doc /* ... */ |
| **attribute** | Typed property or requirement parameter | supportedFormatCount : Integer |
| **require constraint** | Condition the requirement must satisfy | supportedFormatCount >= 2 |
| **requirement usage** | Uses a previously defined requirement | sharedCapture : SharedIdeaCapture |

**TEACHING QUESTION / Which part is formal and checkable, and which part expresses intent in prose?**

SECTION 4.2.4 / LOGICAL STRUCTURE

4 MIN

# Modeling logical structure: the Idea Wall

Definitions and usages describe abstract elements without choosing technologies.

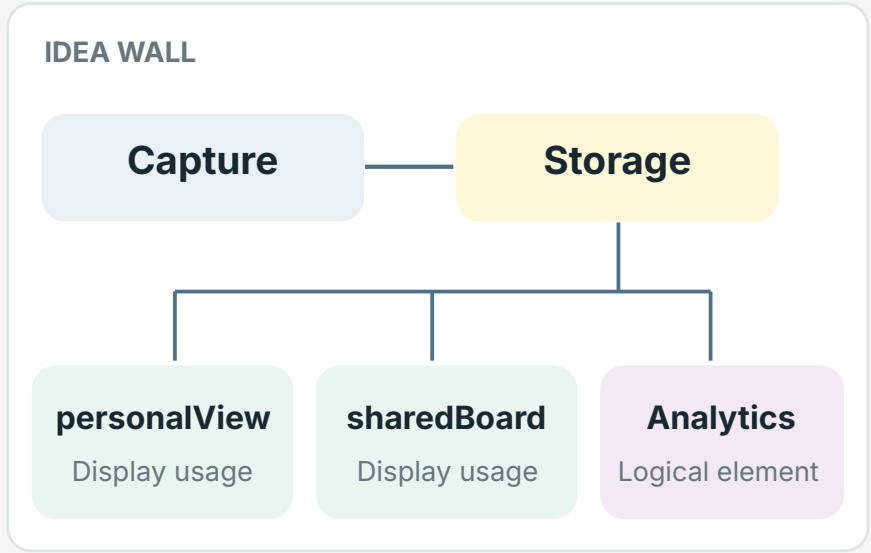
## IDEA WALL / LOGICAL STRUCTURE

```
package IdeaWallStructure {
  part def Capture;
  part def Storage;
  part def Display;
  part def Analytics;

  part def IdeaWall {
    part capture : Capture;
    part storage : Storage;
    part personalView : Display;
    part sharedBoard : Display;
    part analytics : Analytics;

    connect capture to storage;
    connect storage to personalView;
    connect storage to sharedBoard;
    connect storage to analytics;
  }
  part ideaWall : IdeaWall;
}
```

## CONCEPTUAL ASSEMBLY




Connections are undirected; data flow is modeled separately.

SECTION 4.2.5 / STRUCTURE PRIMITIVES

2 MIN

# SysML v2 primitives: logical structure

Definitions describe kinds of elements; usages describe their roles inside a product.

| PRIMITIVE | MEANING | IDEA WALL EXCERPT |
|---|---|---|
| **package** | Groups related elements | package IdeaWallStructure |
| **part def** | Defines a reusable logical or physical element | part def Display; |
| **part** | Declares an element usage in context | part sharedBoard : Display; |
| **:** | Specifies the type of a usage | sharedBoard : Display |
| **nested part** | Builds the product composition | part storage : Storage; |
| **connect … to …** | Establishes a structural connection | connect capture to storage; |

**DEFINITION** part def Display;

**USAGE** part sharedBoard : Display;

SECTION 4.2.6 / BEHAVIOR

4 MIN

# Modeling behavior: capture an idea

Actions express what happens and which activities must follow others.

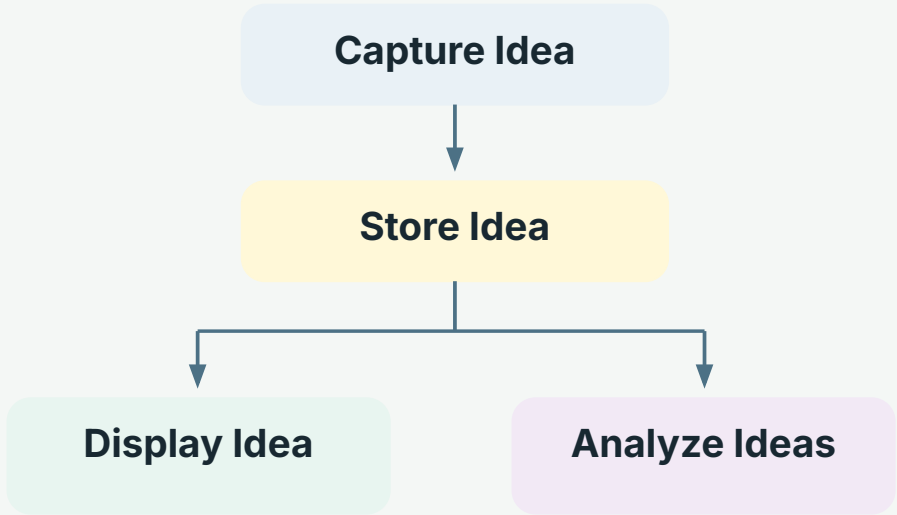
**IDEA WALL / ACTION MODEL**

```
package IdeaWallBehavior {
  action def CaptureIdea;
  action def StoreIdea;
  action def DisplayIdea;
  action def AnalyzeIdeas;

  action def ProcessIdea {
    action capture : CaptureIdea;
    action store : StoreIdea;
    action display : DisplayIdea;
    action analyze : AnalyzeIdeas;

    first capture then store;
    first store then display;
    first store then analyze;
  }
  action processIdea : ProcessIdea;
}
```

**SIMPLIFIED BEHAVIOR**




Both follow Store; their relative order is unspecified.

SECTION 4.2.7 / BEHAVIOR PRIMITIVES

2 MIN

# SysML v2 primitives: behavior

Actions are reusable behaviors; successions establish their required ordering.

| PRIMITIVE | MEANING | IDEA WALL EXCERPT |
|---|---|---|
| **action def** | Defines a reusable behavior | action def CaptureIdea; |
| **action** | Declares a usage of a behavior | action capture : CaptureIdea; |
| **:** | Types the usage by its definition | capture : CaptureIdea |
| **nested action** | Decomposes a larger process | action store : StoreIdea; |
| **first ... then** | Specifies an ordering constraint | first capture then store; |
| **top-level usage** | Introduces an action in the package | action processIdea : ProcessIdea; |

REMEMBER **Successions establish order; they do not by themselves transfer data or guarantee parallelism.**

SECTION 4.2.8 / MINIMAL SPECIFICATION

4 MIN

# From a logical element to a minimal specification

Describe required capabilities and limits, while leaving the implementation open.

BEFORE / LOGICAL ELEMENT

```
part def Display;
```

**What kind of element is this?**

The definition says little about its capabilities.

AFTER / MINIMALLY SPECIFIED DISPLAY

```
part def Display {
  doc /* Present ideas and refresh the view. */
  attribute visibleIdeaCapacity : Integer;
  attribute updateLatencySeconds : Real;
  perform action displayIdea :
    IdeaWallBehavior::DisplayIdea;
  assert constraint {
    visibleIdeaCapacity >= 10 and
    updateLatencySeconds <= 2.0
  }
}
```

KEY IDEA **Minimal does not mean vague: specify WHAT must be achieved, not HOW to implement it.**

SECTION 4.2.9 / SPECIFICATION PRIMITIVES

2 MIN

# SysML v2 primitives: specifications

A minimal specification combines existing constructs rather than introducing a new one.

| PRIMITIVE | MEANING | IDEA WALL EXCERPT |
|---|---|---|
| **doc** | States the intended responsibility | doc /* Present ideas ... */ |
| **attribute** | Declares a property that can be constrained | visibleIdeaCapacity : Integer |
| **Integer / Real** | Scalar value types from the library | Integer; Real |
| **perform action** | Links a part to behavior it can perform | displayIdea : DisplayIdea |
| **assert constraint** | States a condition the element must obey | visibleIdeaCapacity >= 10 |
| **>= / <= / and** | Comparison and logical operators | latency <= 2.0 |
| **::** | Qualifies a name across namespaces | IdeaWallBehavior::DisplayIdea |

**Day 4 OUTPUT → an abstract concept with enough detail to compare realizations on Day 4**

DAY 4 / END

# Today, we modeled what the product should do.

## SysML v2 makes user intent and design logic visible.

A concept model gives us a basis for the next design decisions.

**Questions?**

TODAY'S MODEL MAKES VISIBLE

01 **USER NEED**

Who the product serves

02 **PRODUCT INTENT**

What the product should do

03 **DESIGN LOGIC**

Requirements, structure, behavior